

DEPARTMENT OF MECHANICAL ENGINEERING

MECHATRONICS & IOT 24MEC51

EXERCISE NO: 9

GURUTHARSAN S	- 24MER011
HARISH KUMAR K K	- 24MER015
MADHANRAJ K	- 24MER028
SADAAGOPAAN T R	- 24MER052

MECHATRONICS & IoT

ASSIGNMENT REPORT – 9

TITLE:

Design and develop a Worker Health and Safety Monitoring System for Construction Industries. The system monitors worker safety parameters and generates alerts during unsafe working conditions.

Done By:

GURTHARSAN S	- 24MER011
HARISH KUMAR K K	– 24MER015
MADHANRAJ K	– 24MER028
SADAAGOPAAN T R	– 24MER052

Project Overview:

Construction workers are exposed to several hazards such as excessive temperature, toxic gases, falls, high noise levels, abnormal heart rate, and unsafe environmental conditions. Continuous manual monitoring of these parameters is difficult, especially at large construction sites.

The proposed Worker Health and Safety Monitoring System is an IoT-based wearable safety system designed to continuously monitor the health condition of workers and their surrounding environment. Sensors attached to the worker collect parameters such as heart rate, body temperature, environmental temperature/humidity, gas concentration, and fall/impact conditions.

A microcontroller processes the sensor data and compares it with predefined safety limits. If an unsafe condition is detected, the system immediately activates a buzzer and warning LED and can transmit an alert to a monitoring station or cloud platform.

Problem Statement

Construction sites contain many potential hazards. Workers may experience accidents because of falls, excessive heat, toxic gases, fatigue, or sudden health problems. Conventional safety systems generally depend on supervisors and periodic inspections.

These approaches have several limitations:

- A supervisor cannot continuously monitor every worker.
- Health abnormalities may not be detected immediately.
- Falls may occur in isolated areas where assistance is delayed.
- Hazardous gases can accumulate without being noticed.
- High temperature and humidity can cause heat stress.
- Manual monitoring does not provide continuous digital records.
- Emergency response can be delayed.

Proposed Solution

The proposed system consists of a wearable electronic unit installed on a worker's helmet, safety vest, wristband, or belt.

The ESP32 acts as the central controller. Sensor readings are periodically collected and compared with safety thresholds.

If the measured value is within the safe range, the system continues normal monitoring. If a dangerous condition is detected, the controller.

When an unsafe condition such as a fall, abnormal heart rate, excessive temperature, or hazardous gas level is detected, the system automatically activates an alarm and sends an alert to the monitoring station for immediate action.

System Architecture

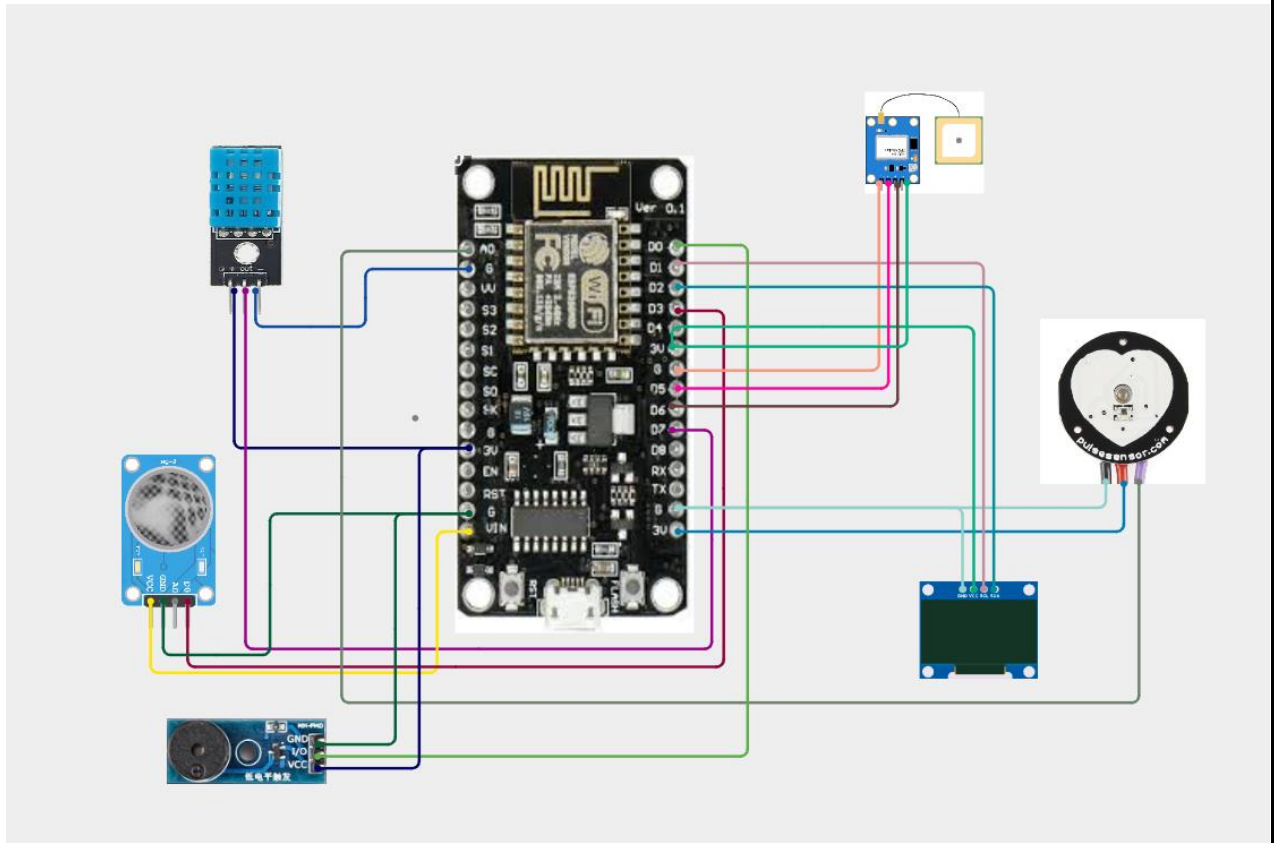
WORKER HEALTH AND SAFETY MONITORING SYSTEM ARCHITECTURE FOR CONSTRUCTION



Circuit Table

Component	Pin	ESP32 Connection
MAX30102	VCC	3.3 V
MAX30102	GND	GND
MAX30102	SDA	GPIO 21
MAX30102	SCL	GPIO 22
MPU6050	VCC	3.3 V
MPU6050	GND	GND
MPU6050	SDA	GPIO 21
MPU6050	SCL	GPIO 22
DHT22	VCC	3.3 V
DHT22	GND	GND
DHT22	DATA	GPIO 4
DS18B20	VCC	3.3 V
DS18B20	GND	GND
DS18B20	DATA	GPIO 5
MQ sensor	VCC	5 V
MQ sensor	GND	GND
MQ sensor	AO	GPIO 34*
Buzzer	+	GPIO 25
Buzzer	-	GND
LED	Anode	GPIO 26 through resistor
LED	Cathode	GND
OLED	SDA	GPIO 21
OLED	SCL	GPIO 22
GPS	TX	ESP32 RX
GPS	RX	ESP32 TX

Circuit Design



Algorithm

1. Start the system.
2. Initialize the ESP32 and all connected sensors.
3. Read the worker's heart rate and body temperature.
4. Read the environmental temperature, humidity, and gas level.
5. Read the motion/acceleration data to detect falls.
6. Compare all sensor values with the predefined safety thresholds.
7. If all values are normal, display "SAFE" and continue monitoring.
8. If any value exceeds the safe limit, activate the buzzer and warning LED.
9. Send an emergency alert and worker location to the monitoring system.
10. Continue monitoring the worker's condition.
11. Repeat the process until the system is switched off.

Coding

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <TinyGPSPlus.h>
#include <SoftwareSerial.h>
#include <DHT.h>

#define WIFI_SSID "Harish Kumar K K"
#define WIFI_PASS "Mavboiii@10"

#define IO_USERNAME "Jr_Frost"
#define IO_KEY "aio_obVD87ub3BbEG4osjIOQjHNrUn7m"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);

AdafruitIO_Feed *tempFeed    = io.feed("temperature");
AdafruitIO_Feed *humFeed     = io.feed("humidity");
AdafruitIO_Feed *pulseFeed   = io.feed("heart-rate");
AdafruitIO_Feed *gasFeed     = io.feed("gas-status");
AdafruitIO_Feed *alertFeed   = io.feed("safety-alert");

AdafruitIO_Feed *latitudeFeed = io.feed("latitude");
AdafruitIO_Feed *longitudeFeed = io.feed("longitude");
AdafruitIO_Feed *locationFeed = io.feed("gps-location");

#define PULSE_PIN  A0
#define GAS_PIN    5
#define DHT_PIN    4
```

```
#define BUZZER_PIN 13
#define DHTTYPE DHT11

SoftwareSerial gpsSerial(14, 12);
TinyGPSPlus gps;
DHT dht(DHT_PIN, DHTTYPE);

const unsigned long uploadInterval = 20000;
unsigned long lastUpload = 0;

int BPM = 0;
bool beatDetected = false;
const int threshold = 550;
unsigned long lastBeatTime = 0;

void readPulse() {

    int signal = analogRead(PULSE_PIN);

    if (signal > threshold && !beatDetected) {

        beatDetected = true;

        unsigned long now = millis();

        if (lastBeatTime != 0)
            BPM = 60000 / (now - lastBeatTime);

        lastBeatTime = now;
    }
}
```



```
    if (signal < threshold - 40)
        beatDetected = false;
}

void setup() {

    Serial.begin(115200);
    gpsSerial.begin(9600);

    dht.begin();

    pinMode(GAS_PIN, INPUT);
    pinMode(BUZZER_PIN, OUTPUT);

    digitalWrite(BUZZER_PIN, HIGH);

    Serial.println();
    Serial.println("=====");
    Serial.println(" WORKER HEALTH & SAFETY SYSTEM");
    Serial.println("=====");

    Serial.print("Connecting to Adafruit IO");

    io.connect();

    while (io.status() < AIO_CONNECTED) {
        Serial.print(".");
        delay(500);
    }
```

```
Serial.println("\nConnected Successfully!");
Serial.println("Waiting for GPS Fix...");
}

void loop() {

    io.run();

    readPulse();

    while (gpsSerial.available()) {
        gps.encode(gpsSerial.read());
    }

    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

    bool gasDetected = (digitalRead(GAS_PIN) == LOW);

    bool danger = false;
    String alertMessage = "";

    if (!isnan(temperature) && temperature > 40) {
        danger = true;
        alertMessage += "HIGH TEMP ";
    }

    if (gasDetected) {
        danger = true;
        alertMessage += "GAS LEAK ";
    }
}
```

```
    if (BPM > 120 || (BPM > 0 && BPM < 50)) {  
        danger = true;  
        alertMessage += "ABNORMAL PULSE ";  
    }
```

```
    if (danger)  
        digitalWrite(BUZZER_PIN, LOW);  
    else  
        digitalWrite(BUZZER_PIN, HIGH);
```

```
    if (millis() - lastUpload >= uploadInterval) {
```

```
        lastUpload = millis();
```

```
        Serial.println("-----");
```

```
        Serial.print("Temperature : ");
```

```
        Serial.print(temperature);
```

```
        Serial.println(" C");
```

```
        Serial.print("Humidity   : ");
```

```
        Serial.print(humidity);
```

```
        Serial.println(" %");
```

```
        Serial.print("Heart Rate : ");
```

```
        Serial.print(BPM);
```

```
        Serial.println(" BPM");
```

```
        Serial.print("Gas Status : ");
```

```
Serial.println(gasDetected ? "DETECTED" : "SAFE");
```

```
tempFeed->save(temperature);
```

```
delay(1200);
```

```
humFeed->save(humidity);
```

```
delay(1200);
```

```
pulseFeed->save(BPM);
```

```
delay(1200);
```

```
gasFeed->save(gasDetected ? 1 : 0);
```

```
delay(1200);
```

```
alertFeed->save(danger ? alertMessage : "SAFE");
```

```
delay(1200);
```

```
if (gps.location.isValid()) {
```

```
    double latitude = gps.location.lat();
```

```
    double longitude = gps.location.lng();
```

```
    double altitude = gps.altitude.isValid() ? gps.altitude.meters() : 0;
```

```
    Serial.print("Latitude : ");
```

```
    Serial.println(latitude, 6);
```

```
    Serial.print("Longitude : ");
```

```
    Serial.println(longitude, 6);
```

```
latitudeFeed->save(latitude);
```

```
delay(1200);
```

```
longitudeFeed->save(longitude);
```

```
delay(1200);
```

```
locationFeed->save(0, latitude, longitude, altitude);
```

```
delay(1200);
```

```
Serial.println("GPS Uploaded Successfully.");
```

```
} else {
```

```
Serial.println("Waiting for GPS satellites...");
```

```
}
```

```
if (danger) {
```

```
Serial.print("ALERT : ");
```

```
Serial.println(alertMessage);
```

```
} else {
```

```
Serial.println("Worker Status : SAFE");
```

```
}
```

```
Serial.println("-----");
```

```
}
```

```
}
```

System Working

Step 1 — System startup

When the device is powered on, the ESP32 initializes all connected sensors and communication modules.

Step 2 — Data acquisition

The sensors continuously collect worker and environmental information.

For example:

- MAX30102 → heart rate
- DS18B20 → body temperature
- DHT22 → ambient temperature and humidity
- MQ sensor → gas/smoke indication
- MPU6050 → acceleration and movement

Step 3 — Data processing

The ESP32 processes the sensor readings and filters abnormal/noisy values where appropriate.

Step 4 — Safety evaluation

The processed readings are compared with configured thresholds.

Step 5 — Local warning

If an unsafe condition occurs, the buzzer and warning LED are activated.

Step 6 — Remote alert

The ESP32 can send the worker's status to a server through Wi-Fi. A GSM/LTE module can be used where Wi-Fi is unavailable.

Bills Of Material

S.No.	Component	Quantity
1	ESP32 development board	1
2	MAX30102	1
3	MPU6050	1
4	DHT22	1
5	DS18B20	1
6	MQ-2/MQ-135 module	1
7	OLED 0.96-inch display	1
8	Buzzer	1
9	LEDs	2–3
10	GPS module	1
11	Rechargeable battery	1
12	Battery/charging module	1
13	Resistors, wires and connectors	1 set
14	Prototype PCB/breadboard	1
15	Safety helmet/holder/enclosure	1

Prototype Implementation

The prototype can be implemented in the following stages.
Hardware implementation

1. Mount the ESP32 on a prototype board.
2. Connect the MAX30102 using the I²C interface.
3. Connect the MPU6050 to the same I²C bus.
4. Connect the DHT22 and DS18B20 to their respective GPIO pins.
5. Connect the MQ sensor to an ESP32 ADC input through appropriate voltage protection/scaling.
6. Connect the buzzer and warning LED.
7. Add the OLED display for local status information.
8. Add GPS if location tracking is required.
9. Install the electronics in a protective enclosure.
10. Attach the enclosure to a helmet, vest, belt or wearable band.

Results & Performance Analysis

The prototype should be evaluated under controlled test conditions before deployment at a real construction site.

Test condition	Expected system response
Normal heart rate	No alarm
Abnormally high/low heart rate	Warning
Elevated body temperature	Warning
High ambient temperature	Warning
High gas-sensor reading	Emergency warning
Simulated fall	Fall alarm
Normal movement	No fall alarm
Communication available	Data transmitted
Communication unavailable	Local alarm continues

Performance Analysis

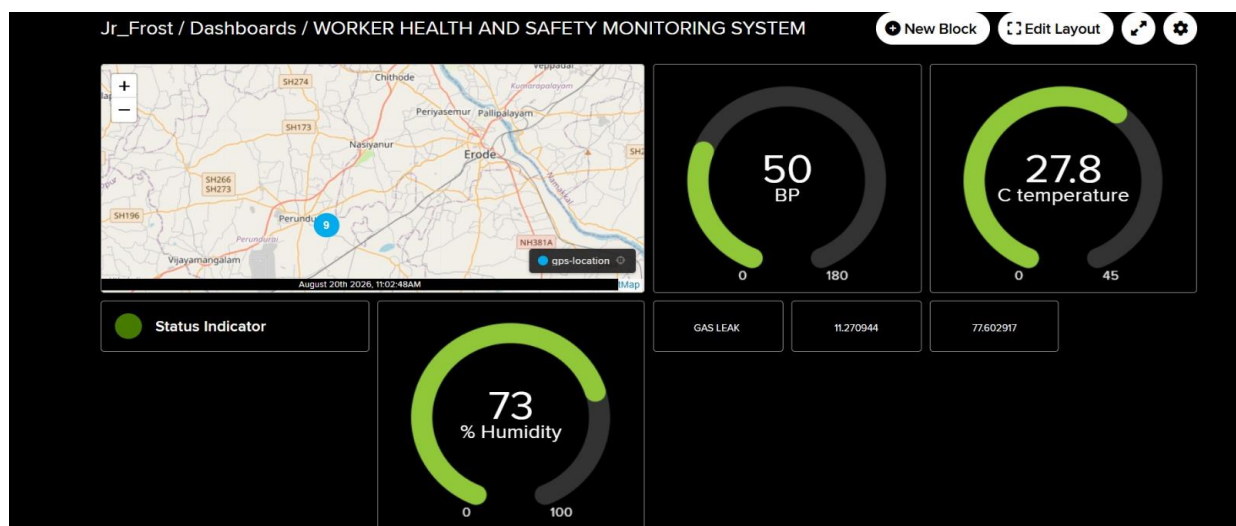
The system continuously monitors worker health and environmental conditions with quick alert generation. It provides reliable detection of unsafe conditions such as abnormal temperature, gas levels, and falls, improving worker safety and enabling faster emergency response.

```
Gas Status : SAFE
Latitude : 11.270944
Longitude : 77.602917
Uploading to Adafruit IO...
Upload Successful.
Worker Status : SAFE
```

```
-----
Temperature : 27.8 C
Humidity : 72.8 %
Heart Rate : 88 BPM
Gas Status : SAFE
Latitude : 11.270944
Longitude : 77.602917
Uploading to Adafruit IO...
Upload Successful.
Worker Status : SAFE
```

```
-----
Temperature : 27.8 C
Humidity : 72.9 %
Heart Rate : 76 BPM
Gas Status : SAFE
Latitude : 11.270944
Longitude : 77.602917
Uploading to Adafruit IO...
Upload Successful.
Worker Status : SAFE
```

```
-----
Temperature : 27.9 C
Humidity : 72.9 %
Heart Rate : 53 BPM
Gas Status : SAFE
Latitude : 11.270944
Longitude : 77.602917
Uploading to Adafruit IO...
```



```
-----
Upload Successful.
Worker Status : SAFE
-----

Temperature : 27.7 C
Humidity : 73.3 %
Heart Rate : 59 BPM
Gas Status : SAFE
Latitude : 11.270944
Longitude : 77.602917
Uploading to Adafruit IO...
Upload Successful.
Worker Status : SAFE
-----

Temperature : 27.8 C
Humidity : 73.1 %
Heart Rate : 73 BPM
Gas Status : DETECTED
Latitude : 11.270944
Longitude : 77.602917
Uploading to Adafruit IO...
Upload Successful.
ALERT : GAS LEAK
-----

Temperature : 27.8 C
Humidity : 72.9 %
Heart Rate : 40 BPM
Gas Status : DETECTED
Latitude : 11.270944
Longitude : 77.602917
Uploading to Adafruit IO...
Upload Successful.
ALERT : GAS LEAK ABNORMAL PULSE
-----
```

